

CSE 309 (Compilers)

Basic Concepts

Dr. Muhammad Masroor Ali

Professor

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka-1205, Bangladesh

January 2025

Adapted by
Ahmed Mahir Sultan Rumi
Lecturer, Department of CSE, BUET

Version: 1.2, Last modified: June 20, 2026

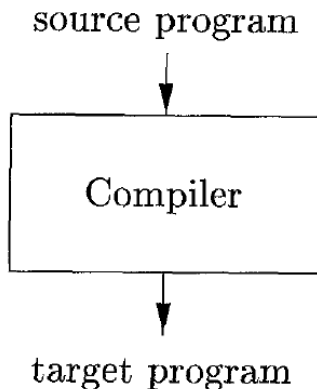
Introduction

- Programming languages are notations for describing computations to people and to machines.
- The world as we know it depends on programming languages, because all the software running on all the computers was written in some programming language.
- But, before a program can be run, it first must be translated into a form in which it can be executed by a computer.
- The software systems that do this translation are called compilers.

Introduction

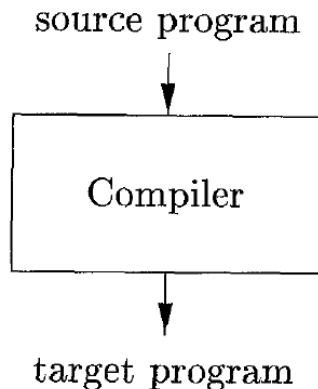
- Programming languages are notations for describing computations to people and to machines.
- The world as we know it depends on programming languages, because all the software running on all the computers was written in some programming language.
- But, before a program can be run, it first must be translated into a form in which it can be executed by a computer.
- The software systems that do this translation are called compilers.

- Simply stated, a compiler is a program,
 - that can read a program in one language — the source language — and
 - translate it into an equivalent program in another language — the target language.



A compiler

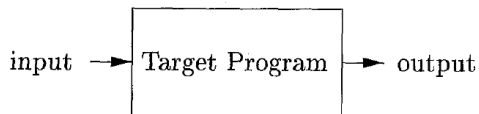
- An important role of the compiler is to report any errors in the source program that it detects during the translation process.



A compiler

Language Processors

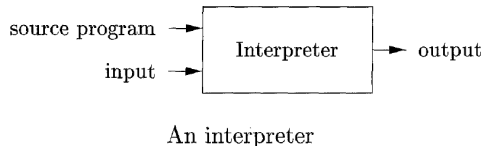
- If the target program is an executable machine-language program, it can then be called by the user to process inputs and produce outputs.



Running the target program

Language Processors — *continued*

- An interpreter is another common kind of language processor.
- Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user.

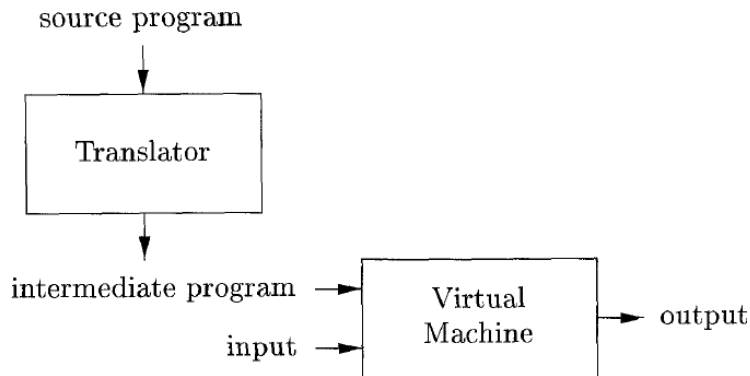


- The machine-language target program produced by a compiler is usually much faster than an interpreter at mapping inputs to outputs.
- An interpreter, however, can usually give better error diagnostics than a compiler, because it executes the source program statement by statement.

- Then which path do the modern compilers take?
- Obvious answer. The middle way.
 - Hybrid compilers
 - Just-in-time compilers (JIT)

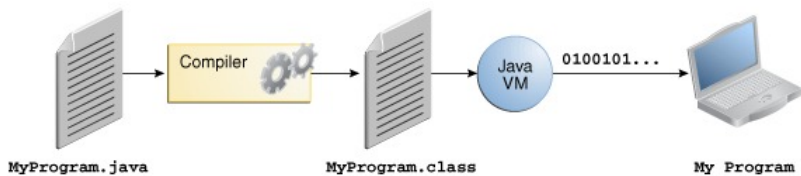
- Then which path do the modern compilers take?
- Obvious answer. The middle way.
 - Hybrid compilers
 - Just-in-time compilers (JIT)

Example



A hybrid compiler

- Java language processors combine compilation and interpretation.



Java Program

```
class HelloWorldApp {  
    public static void main(string [] args) {  
        System.out.println("Hello World!");  
    }  
}
```

HelloWorldApp.java

Compiler

JVM

JVM

JVM

Win32

UNIX

MacOS

Example — *continued*

- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- "Write once, run anywhere"

Example — *continued*

- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- "Write once, run anywhere" A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.

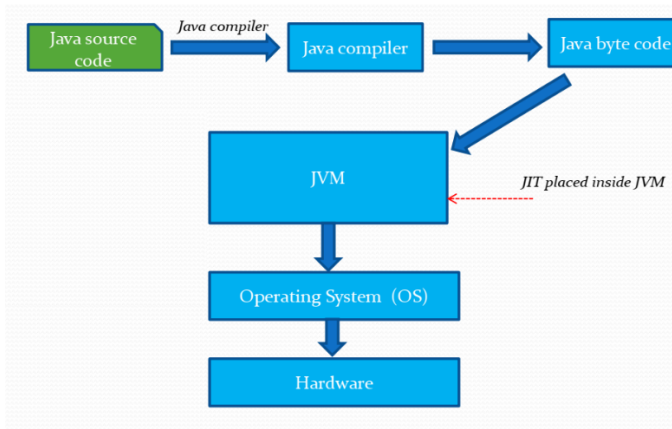
Example — *continued*

- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- "Write once, run anywhere" A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.
- Faster and better performance than pure interpretation

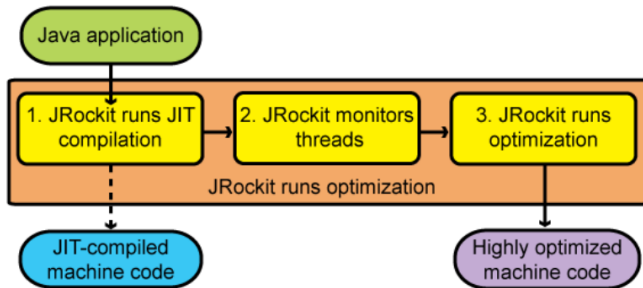
Example — *continued*

- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- "Write once, run anywhere" A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.
- Faster and better performance than pure interpretation
- One important point. You will need JVM in the machine to run bytecode. That is platform dependent.

Just-in-Time Compilation

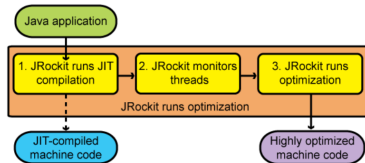


Just-in-Time Compilation - JRockit — *continued*



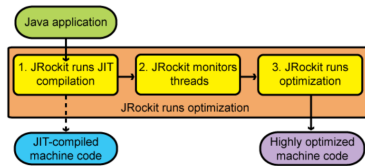
JRockit Just-in-Time Compilation

- When a section of instructions is called:
 - Compile bytecode into machine code just in time.
 - Run the compiled machine code.
 - The compiled code is not fully optimized.
 - It may be slower than bytecode interpretation.
 - JVM startup may be slower than execution.



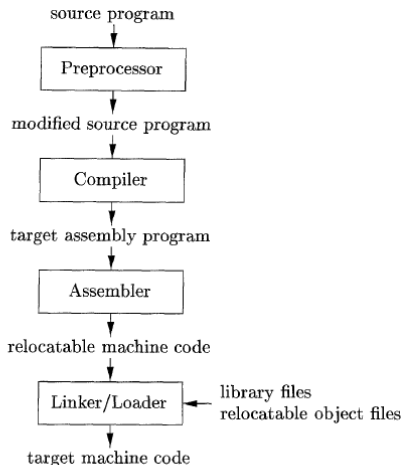
JRockit Optimization

- Identify which functions merit optimization:
 - A sampler thread checks the status of active threads.
 - Hot methods are earmarked for optimization.
 - Optimization opportunities occur early on.
- In the background, compile optimized hot methods into machine-readable instructions.



Language Processors — *continued*

- In addition to a compiler, several other programs may be required to create an executable target program.
- A source program may be divided into modules stored in separate files.
- The task of collecting the source program is sometimes entrusted to a separate program, called a preprocessor.
- The preprocessor may also expand shorthands, called macros, into source language statements.



A language-processing system

C Preprocessor

- The C preprocessor or `cpp` is the macro preprocessor for the C and C++ computer programming languages.
- The preprocessor provides the ability for:
 - the inclusion of header files,
 - macro expansions,
 - conditional compilation, and
 - line control.

- In many C implementations, it is a separate program invoked by the compiler as the first part of translation.

C Preprocessor — Including Files

- One of the most common uses of the preprocessor is to include another file:

```
#include <stdio.h>
int main(void)
{
    printf("Hello, _world!\n");
    return 0;
}
```

C Preprocessor — Including Files

```
#include <stdio.h>
int main(void)
{
    printf("Hello, _world!\n");
    return 0;
}
```

- The preprocessor replaces the line `#include <stdio.h>` with the text of the file 'stdio.h'.

C Preprocessor — Conditional Compilation

- The `if-else` directives `#if`, `#ifdef`, `#ifndef`, `#else`, `#elif` and `#endif` can be used for conditional compilation.
- What is the difference between `if` and `#if`?

Difference Between `#if` and `if`

- The main difference is when the condition is checked.
- `#if` is a preprocessor-time condition.
- `if` is a runtime condition.
- Therefore, `#if` decides whether code reaches the compiler.
- `if` decides whether compiled code executes while the program runs.

`#if`: Preprocessor-Time Condition

```
#define DEBUG 1

#if DEBUG
printf("Debug mode\n");
#endif
```

- The preprocessor handles `#if` before actual compilation.
- If `DEBUG` is 1, the line is kept in the source code.
- If `DEBUG` is 0, the line is removed before compilation.

if: Runtime Condition

```
int x = 5;

if (x > 0) {
    printf("Positive\n");
}
```

- `if` is a normal C/C++ statement.
- It is handled by the compiler and checked at runtime.
- Here, the condition `x > 0` is checked when the program runs.

C Preprocessor — Conditional Compilation — *continued*

- Most compilers targeting Microsoft Windows implicitly define `_WIN32`.
- This allows code, including preprocessor commands, to compile only when targeting Windows systems.

C Preprocessor — Conditional Compilation — *continued*

```
#ifdef _unix_  
/* _unix_ is usually defined by compilers  
   targeting Unix systems */  
# include <unistd.h>  
#elif defined _WIN32  
/* _WIN32 is usually defined by compilers  
   targeting 32 or 64 bit Windows systems */  
# include <windows.h>  
#endif
```

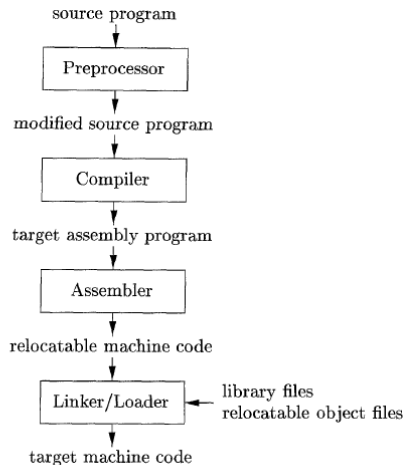
- The example code tests if a macro `_unix_` is defined.
- If it is, the file `<unistd.h>` is then included.
- Otherwise, it tests if a macro `_WIN32` is defined instead.
- If it is, the file `<windows.h>` is then included.

C Preprocessor — Macro Definition and Expansion

```
#define PI 3.14159  
#define RADTODEG(x) ((x) * 57.29578)
```

Language Processors — *continued*

- The modified source program is then fed to a compiler.
- The compiler may produce an assembly-language program as its output, because assembly language is easier to produce as output and is easier to debug.
- The assembly language is then processed by a program called an assembler that produces relocatable machine code as its output.



A language-processing system

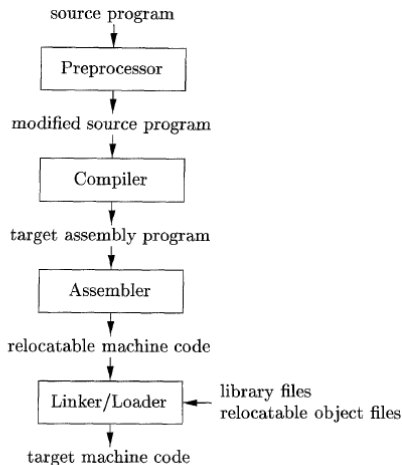
Assembler

<https://cs.stackexchange.com/a/13905/7200>

- An assembler translates assembly code to machine code.
- The translation is mechanical, and can be done in only one way.
- Assembly languages are usually designed to be nearly trivial to parse and type check and tend to involve a lot of table-driven generators.
- Why do we have this assembler step in the pipeline?

Language Processors — *continued*

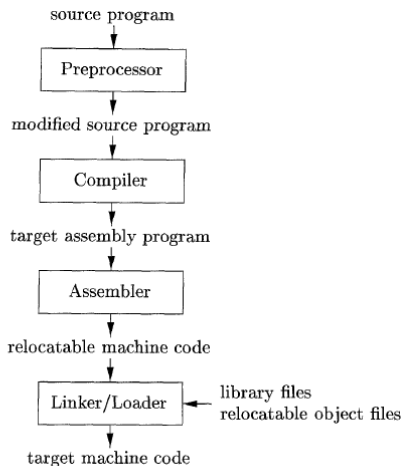
- Large programs are often compiled in pieces.
- So the relocatable machine code may have to be linked together with other relocatable object files and library files into the code that actually runs on the machine.



A language-processing system

Language Processors — *continued*

- The linker resolves external memory addresses, where the code in one file may refer to a location in another file.
- The loader then puts together all of the executable object files into memory for execution.



A language-processing system

Relocatable Machine Code

[https://www.wikiwand.com/en/Relocation_\(computing\)](https://www.wikiwand.com/en/Relocation_(computing)),
<https://stackoverflow.com/a/46101660/1337854> and
[https://gateoverflow.in/18466/
difference-between-relocatable-machine-absolute-machine](https://gateoverflow.in/18466/difference-between-relocatable-machine-absolute-machine)

- Relocation is the process of assigning load addresses for position-dependent code and data of a program and adjusting the code and data to reflect the assigned addresses.
- Relocatable-Code, as the name suggests, is relocatable in that code can be loaded anywhere in memory.
- It usually has to be relocated or fixed up before it is executable.

Relocatable Machine Code — *continued*

- When the machine code is generated then the computer does not know where the code will be placed in RAM till run time.
- So all the lines are numbered from 0-1-2... and so on.
- Now suppose your program is loaded at 230 address then this address on run time will be converted to $230 + 0$, $230 + 1$, $230 + 2$.
- This means every line will be adjusted accordingly.

Relocatable Machine Code — *continued*

- (A) Program SUBR is compiled to produce object file (B), shown as both machine code and assembler.
- The compiler may start the compiled code at an arbitrary location, often location 1 as shown.
- Location 13 contains the machine code for the jump instruction to statement ST in location 5.

<pre>subr: procedure; ... some code ... st: a=0; ... more code ... goto st;</pre> A	
<pre>1: + 0 0 0 0 0 5: + 0 0 0 2 48 13: + 0 5 0 0 39</pre>	<pre>SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... JMP ST</pre> B
<pre>120: + 0 0 0 0 0 125: + 0 0 0 2 48 133: + 1 61 0 0 39</pre>	<pre>SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... JMP ST</pre> C
<pre>300: + 0 0 0 0 0 305: + 0 0 0 2 48 313: + 4 49 0 0 39</pre>	<pre>SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... JMP ST</pre> D

Relocatable Machine Code — *continued*

- (C) If SUBR is later linked with other code it may be stored at a location other than 1.
- In this example the linker places it at location 120.

<pre>subr: procedure; ... some code ... st: a=0; ... more code ... goto st;</pre>	
A	
1: + 0 0 0 0 0 5: + 0 0 0 2 48 13: + 0 5 0 0 39	SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... JMP ST
B	
120: + 0 0 0 0 0 125: + 0 0 0 2 48 133: + 1 61 0 0 39	SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... JMP ST
C	
300: + 0 0 0 0 0 305: + 0 0 0 2 48 313: + 4 49 0 0 39	SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... JMP ST
D	

Relocatable Machine Code — *continued*

- The address in the jump instruction, which is now at location 133, must be relocated to point to the new location of the code for statement ST, now 125.
- 1 61 shown in the instruction is the MIX machine code representation of 125.

<pre>subr: procedure; ... some code ... st: a=0; ... more code ... goto st;</pre> A	
<pre>1: + 0 0 0 0 0 5: + 0 0 0 2 48 13: + 0 5 0 0 39</pre>	<pre>SUBR EQU * ... some code ... ST ENT 0 maybe ... more code ... B JMP ST</pre>
<pre>120: + 0 0 0 0 0 125: + 0 0 0 2 48 133: + 1 61 0 0 39</pre>	<pre>SUBR EQU * ... some code ... ST ENT 0 maybe ... more code ... C JMP ST</pre>
<pre>300: + 0 0 0 0 0 305: + 0 0 0 2 48 313: + 4 49 0 0 39</pre>	<pre>SUBR EQU * ... some code ... ST ENT 0 maybe ... more code ... D JMP ST</pre>

Relocatable Machine Code — *continued*

- (D) When the program is loaded into memory to run it may be loaded at some location other than the one assigned by the linker.
- This example shows SUBR now at location 300.

<pre>subr: procedure; ... some code ... st: a=0; ... more code ... goto st;</pre> A	
1: + 0 0 0 0 0 5: + 0 0 0 2 48 13: + 0 5 0 0 39	SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... B JMP ST
120: + 0 0 0 0 0 125: + 0 0 0 2 48 133: + 1 61 0 0 39	SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... C JMP ST
300: + 0 0 0 0 0 305: + 0 0 0 2 48 313: + 4 49 0 0 39	SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... D JMP ST

Relocatable Machine Code — *continued*

- The address in the jump instruction, now at 313, needs to be relocated again so that it points to the updated location of ST, 305.
- 4 49 is the MIX machine representation of 305.

<pre>subr: procedure; ... some code ... st: a=0; ... more code ... goto st;</pre> A	
<pre>1: + 0 0 0 0 0 5: + 0 0 0 2 48 13: + 0 5 0 0 39</pre>	<pre>SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... B JMP ST</pre>
<pre>120: + 0 0 0 0 0 125: + 0 0 0 2 48 133: + 1 61 0 0 39</pre>	<pre>SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... C JMP ST</pre>
<pre>300: + 0 0 0 0 0 305: + 0 0 0 2 48 313: + 4 49 0 0 39</pre>	<pre>SUBR EQU * ... some code ... ST ENTA 0 maybe ... more code ... D JMP ST</pre>

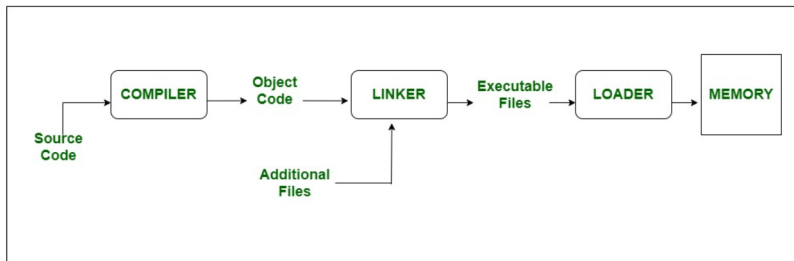
Difference between Linker and Loader

<https://www.geeksforgeeks.org/difference-between-linker-and-loader/>

Linker

- A linker is special program that combines the object files, generated by compiler/assembler, and other pieces of codes to originate an executable file.
- In the object file, linker searches and append all libraries needed for execution of file.
- It regulates memory space that code from each module will hold.
- It also merges two or more separate object programs and establishes link among them.

Difference between Linker and Loader — *continued*



Loader

- The loader is special program that takes input of object code from linker, loads it to main memory, and prepares this code for execution by computer.
- Loader allocates memory space to program.
- Even it settles down symbolic reference between objects.
- It in charge of loading programs and libraries in operating system.

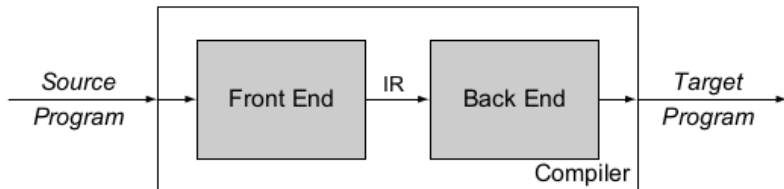
The Structure of a Compiler — *continued*

- To translate text from one language to another, the tool must understand both the form, or syntax, and content, or meaning, of the input language.
- It needs to understand the rules that govern syntax and meaning in the output language.
- Finally, it needs a scheme for mapping content from the source language to the target language.

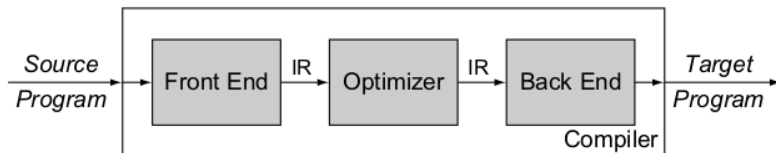
The Structure of a Compiler — *continued*

- The structure of a typical compiler derives from these simple observations.
- The compiler has a front end to deal with the source language.
- It has a back end to deal with the target language.
- Connecting the front end and the back end, it has a formal structure for representing the program in an intermediate form whose meaning is largely independent of either language.
- To improve the translation, a compiler often includes an optimizer that analyzes and rewrites that intermediate form.

The Structure of a Compiler — *continued*



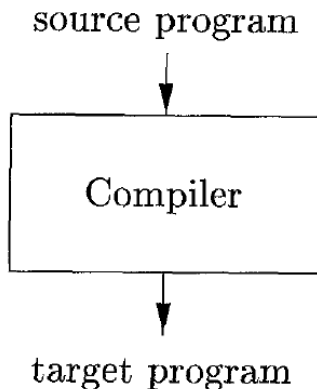
The Structure of a Compiler — *continued*



- What would happen if we didn't separate the Front End and Back End?

The Structure of a Compiler — *continued*

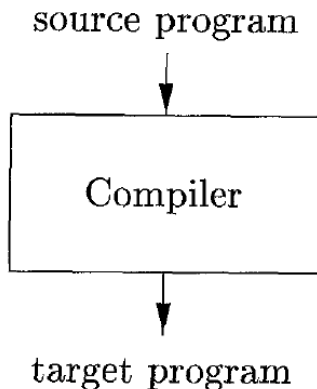
- Up to this point we have treated a compiler as a single box that maps a source program into a semantically equivalent target program.
- If we open up this box a little, we see that there are two parts to this mapping: analysis and synthesis.



A compiler

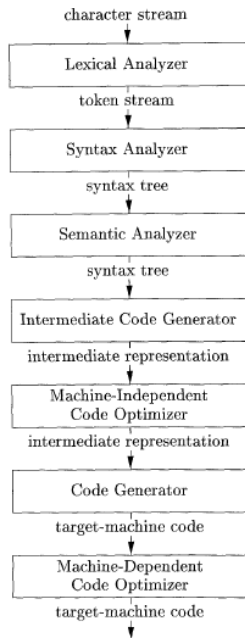
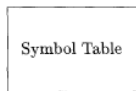
The Structure of a Compiler — *continued*

- Up to this point we have treated a compiler as a single box that maps a source program into a semantically equivalent target program.
- If we open up this box a little, we see that there are two parts to this mapping: analysis and synthesis.

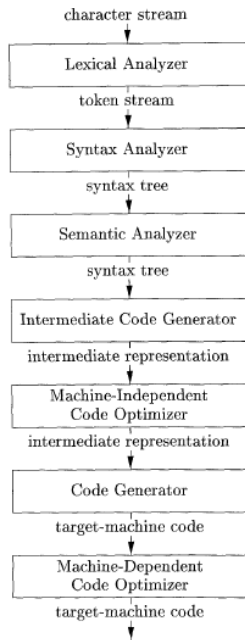
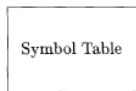


A compiler

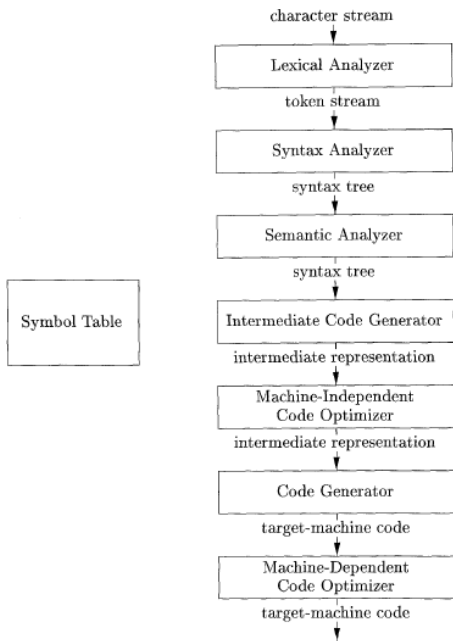
- The analysis part breaks up the source program into constituent pieces and imposes a grammatical structure on them.
- It then uses this structure to create an intermediate representation of the source program.
- If the analysis part detects that the source program is either syntactically ill formed or semantically unsound, then it must provide informative messages, so the user can take corrective action.



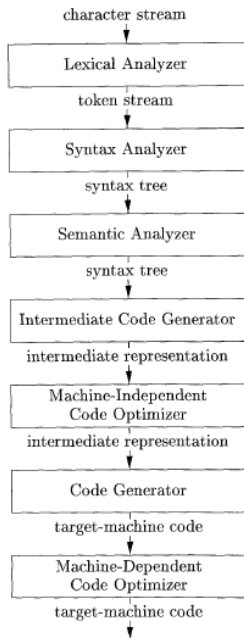
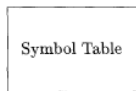
- The analysis part also collects information about the source program and stores it in a data structure called a symbol table, which is passed along with the intermediate representation to the synthesis part.
- The synthesis part constructs the desired target program from the intermediate representation and the information in the symbol table.
- The analysis part is often called the front end of the compiler; the synthesis part is the back end.



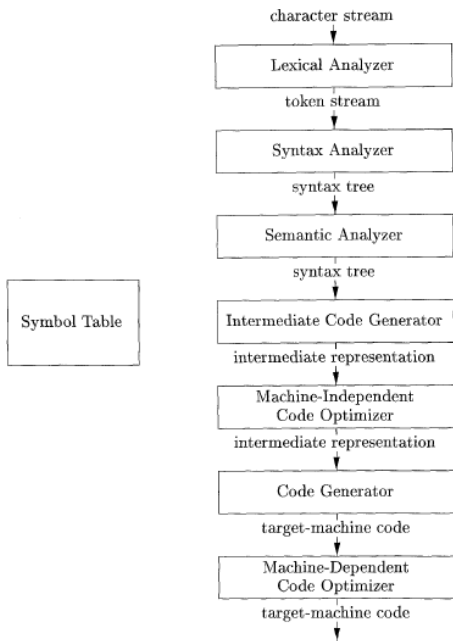
- If we examine the compilation process in more detail, we see that it operates as a sequence of phases, each of which transforms one representation of the source program to another.
- In practice, several phases may be grouped together, and the intermediate representations between the grouped phases need not be constructed explicitly.



- The symbol table, which stores information about the entire source program, is used by all phases of the compiler.

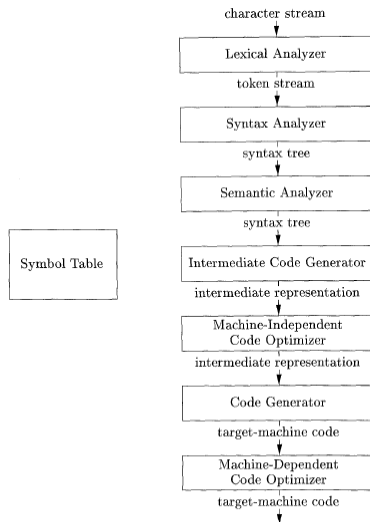


- Some compilers have a machine-independent optimization phase between the front end and the back end.
- The purpose of this optimization phase is to perform transformations on the intermediate representation, so that the back end can produce a better target program than it would have otherwise produced from an unoptimized intermediate representation.
- Since optimization is optional, one or the other of the two optimization phases shown may be missing.



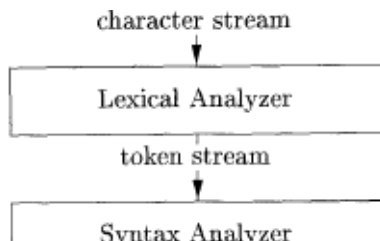
Lexical Analysis

- The first phase of a compiler is called lexical analysis or scanning.



Phases of a compiler

Lexical Analysis — *continued*



- The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called **lexemes**.

Lexical Analysis — *continued*

- For each lexeme, the lexical analyzer produces as output a token of the form

<token-name, attribute-value>

that it passes on to the subsequent phase, syntax analysis.

- In the token, the first component **token-name** is an abstract symbol that is used during syntax analysis, and the second component **attribute-value** points to an entry in the symbol table for this token.
- Information from the symbol-table entry is needed for semantic analysis and code generation.

$\langle \textit{token-name}, \textit{attribute-value} \rangle$

`position = initial + rate * 60`

- We consider a source program that contains the assignment statement
`position = initial + rate * 60`
- The characters in this assignment could be grouped into the lexemes and mapped into the tokens passed on to the syntax analyzer

$\langle \text{token-name}, \text{attribute-value} \rangle$

`position = initial + rate * 60`

1. `position` is a lexeme that would be mapped into a token $\langle \text{id}, 1 \rangle$.
 - **id** is an abstract symbol standing for identifier,
 - 1 points to the symbol-table entry for `position`,
 - The symbol-table entry for an identifier holds information about the identifier, such as its name and type.

$\langle \text{token-name}, \text{attribute-value} \rangle$

`position = initial + rate * 60`

2. The assignment symbol `=` is a lexeme that is mapped into the token $\langle = \rangle$.
- Since this token needs no attribute-value, we have omitted the second component.
 - We could have used any abstract symbol such as `assign` for the token-name, but for notational convenience we have chosen to use the lexeme itself as the name of the abstract symbol.

$\langle \text{token-name}, \text{attribute-value} \rangle$

`position = initial + rate * 60`

3. `initial` is a lexeme that is mapped into the token $\langle \mathbf{id}, 2 \rangle$,
 - 2 points to the symbol-table entry for `initial`.

Lexical Analysis — *continued*

$\langle token-name, attribute-value \rangle$

`position = initial + rate * 60`

4. `+` is a lexeme that is mapped into the token $\langle + \rangle$.

$\langle \text{token-name}, \text{attribute-value} \rangle$

`position = initial + rate * 60`

5. `rate` is a lexeme that is mapped into the token $\langle \text{id}, 3 \rangle$, where 3 points to the symbol-table entry for `rate`.

$\langle \text{token-name}, \text{attribute-value} \rangle$

`position = initial + rate * 60`

6. `*` is a lexeme that is mapped into the token $\langle * \rangle$.

$\langle \text{token-name}, \text{attribute-value} \rangle$

`position = initial + rate * 60`

7. 60 is a lexeme that is mapped into the token $\langle 60 \rangle$.

Lexical Analysis — *continued*

$\langle \text{token-name}, \text{attribute-value} \rangle$

`position = initial + rate * 60`

- Blanks separating the lexemes would be discarded by the lexical analyzer.

Lexical Analysis — *continued*

`position = initial + rate * 60`

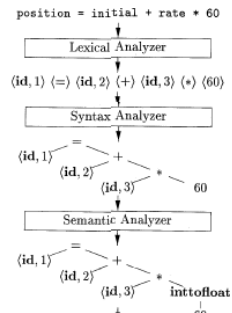
- Figure shows the representation of the assignment statement after lexical analysis as the sequence of tokens,

$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$

- In this representation, the token names =, +, and * are abstract symbols for the assignment, addition, and multiplication operators, respectively.

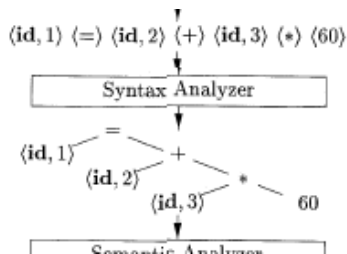
1	position	...
2	initial	...
3	rate	...

SYMBOL TABLE



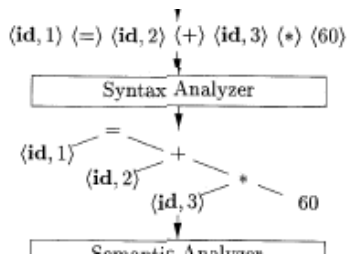
Syntax Analysis

- The second phase of the compiler is syntax analysis or parsing.
- The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.



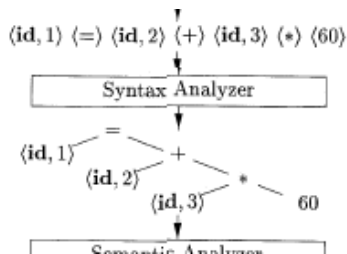
Syntax Analysis

- A typical representation is a syntax tree in which each interior node represents an operation and the children of the node represent the arguments of the operation.



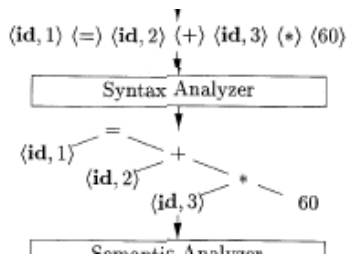
Syntax Analysis

- This tree shows the order in which the operations in the assignment `position = initial + rate * 60` are to be performed.



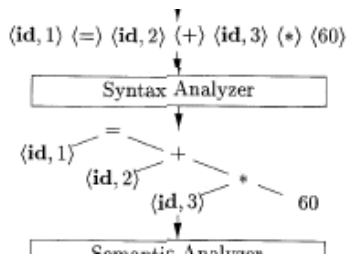
Syntax Analysis

- The tree has an interior node labeled $*$ with $\langle \text{id}, 3 \rangle$ as its left child and the integer 60 as its right child. The node $\langle \text{id}, 3 \rangle$ represents the identifier `rate`.
- The node labeled $*$ makes it explicit that we must first multiply the value of `rate` by 60.



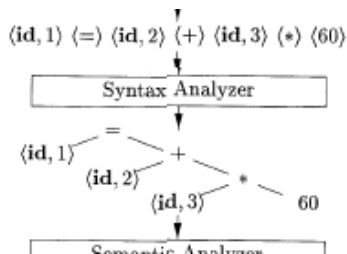
Syntax Analysis

- The node labeled + indicates that we must add the result of this multiplication to the value of `initial`.



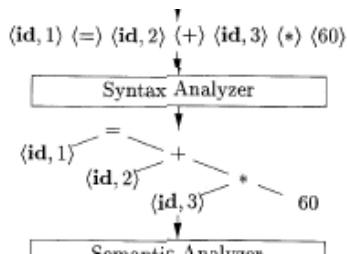
Syntax Analysis

- The root of the tree, labeled =, indicates that we must store the result of this addition into the location for the identifier `position`.



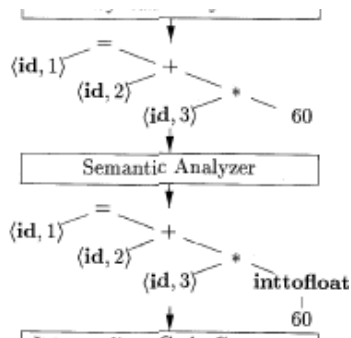
Syntax Analysis

- This ordering of operations is consistent with the usual conventions of arithmetic which tell us that multiplication has higher precedence than addition, and hence that the multiplication is to be performed before the addition.



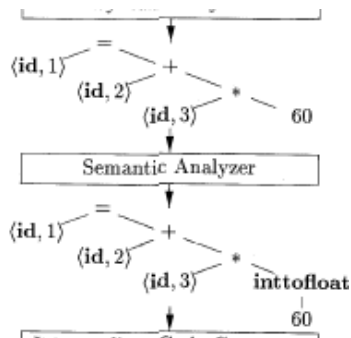
Semantic Analysis

- The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.
- It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.



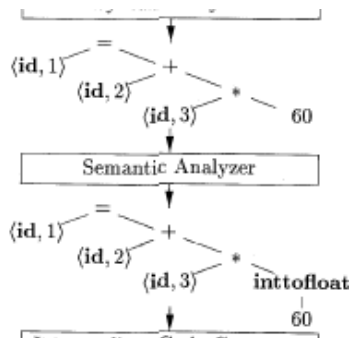
Semantic Analysis

- An important part of semantic analysis is type checking, where the compiler checks that each operator has matching operands.
- For example, many programming language definitions require an array index to be an integer.
- The compiler must report an error if a floating-point number is used to index an array.



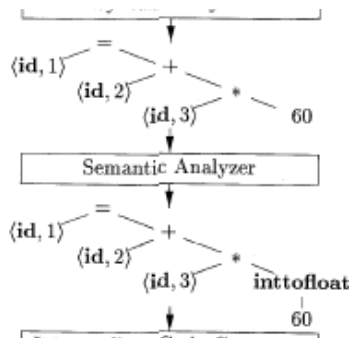
Semantic Analysis

- The language specification may permit some type conversions called coercions.
- For example, a binary arithmetic operator may be applied to either a pair of integers or to a pair of floating-point numbers.
- If the operator is applied to a floating-point number and an integer, the compiler may convert or coerce the integer into a floating-point number.



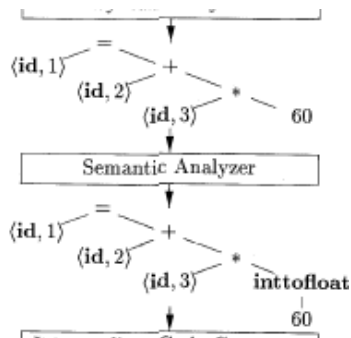
Semantic Analysis

- Suppose that `position`, `initial`, and `rate` have been declared to be floating-point numbers, and that the lexeme `60` by itself forms an integer.
- The type checker in the semantic analyzer discovers that the operator `*` is applied to a floating-point number `rate` and an integer `60`.
- In this case, the integer may be converted into a floating-point number.



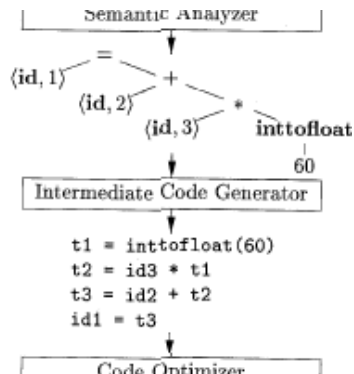
Semantic Analysis

- Notice that the output of the semantic analyzer has an extra node for the operator `inttofloat`, which explicitly converts its integer argument into a floating-point number.



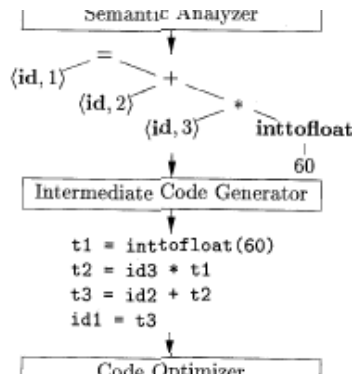
Intermediate Code Generation

- In the process of translating a source program into target code, a compiler may construct one or more intermediate representations, which can have a variety of forms.
- Syntax trees are a form of intermediate representation.
- They are commonly used during syntax and semantic analysis.



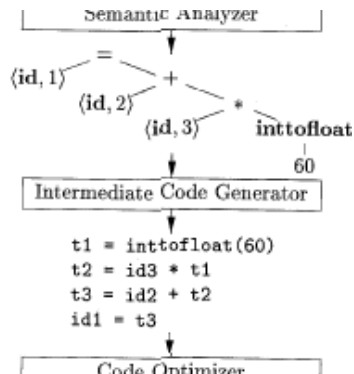
Intermediate Code Generation

- After syntax and semantic analysis of the source program, many compilers generate an explicit low-level or machine-like intermediate representation, which we can think of as a program for an abstract machine.
- This intermediate representation should have two important properties:
 - it should be easy to produce and
 - it should be easy to translate into the target machine.



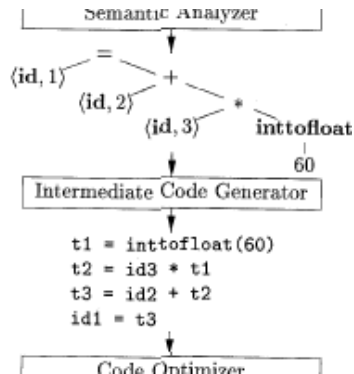
Intermediate Code Generation

- We consider an intermediate form called “three-address code”.
- Like the assembly language for a machine in which every memory location can act like a register.
- Three-address code consists of a sequence of instructions, each of which has at most three operands.



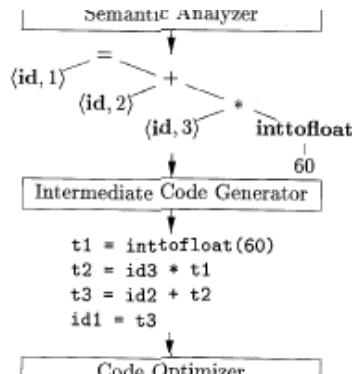
Intermediate Code Generation

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```



Intermediate Code Generation

- Each three-address instruction has at most one operator in addition to the assignment.
 - Has to decide on the order in which operations are to be done.
 - Multiplication precedes the addition in the source program.
- Must generate a temporary name to hold the value computed by each instruction.
- Some “three-address” instructions have fewer than three operands.



Intermediate Codes in Modern Compilers

- In real modern compilers, intermediate code often appears in more advanced forms.
- Examples include SSA form and LLVM IR.
- Three-address code is still relevant because it teaches the core idea of intermediate representation.
- It shows how complex source programs can be represented using simpler compiler-friendly instructions.

Examples of Intermediate Representations

- LLVM IR
 - GIMPLE in GCC
 - MIR in the Rust compiler
 - Java bytecode
 - .NET IL
 - SSA-based IR
- These representations are usually richer than basic three-address code.

Compiler IR: Independence Properties

Machine independent It is not tied to a specific CPU such as x86 or ARM.

Language independent Different source languages can be translated into a similar IR.

Can represent constructs Assignments, expressions, loops, conditionals, function calls, array access, and other constructs can be represented.

Compiler IR: Simple Instructions

- Complex expressions are broken into small operations.
- Each instruction usually has at most one operator.
- Temporary variables store intermediate results.

Example

```
t1 = a + b
t2 = t1 * c
x  = t2
```

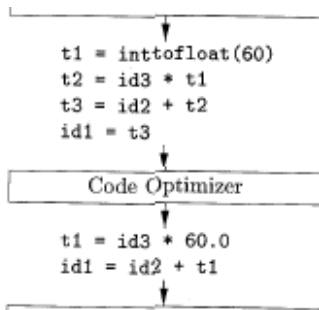
- This is preferred over a complex expression such as $x = (a + b) * c$.

Compiler IR: Control Flow and Optimization

- Branches, jumps, and labels are represented explicitly.
- This makes the control flow of the program clearer to the compiler.
- Optimizations become easier, for example:
 - dead code elimination,
 - constant folding,
 - common subexpression elimination.
- Since the instructions are simple, translating IR to machine code also becomes easier.

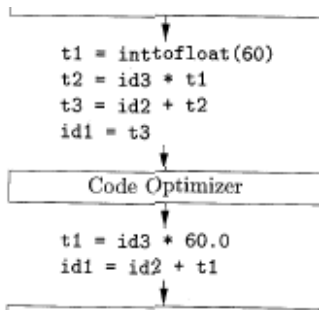
Code Optimization

- The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result.
- Usually better means faster, but other objectives may be desired, such as shorter code, or target code that consumes less power.



Code Optimization

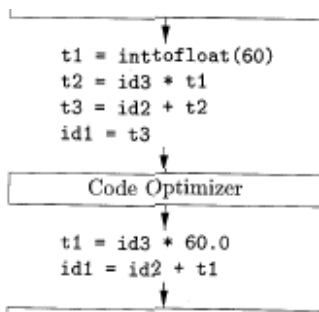
- A simple intermediate code generation algorithm followed by code optimization is a reasonable way to generate good target code.
- The optimizer can deduce that the conversion of 60 from integer to floating point can be done once and for all at compile time.
- So the `inttfloat` operation can be eliminated by replacing the integer 60 by the floating-point number 60.0.
- Moreover, `t3` is used only once to transmit its value to `<id, 1>`.



Code Optimization

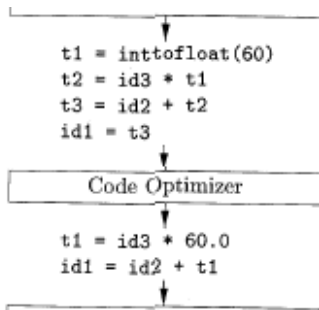
- So the optimizer can transform the code into the shorter sequence,

```
t1 = id3 * 60.0  
id1 = id2 + t1
```



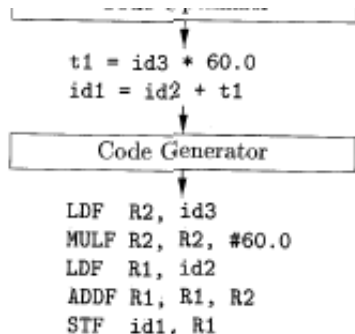
Code Optimization

- There is a great variation in the amount of code optimization different compilers perform.
- In those that do the most, the so-called “optimizing compilers,” a significant amount of time is spent on this phase.
- There are simple optimizations that significantly improve the running time of the target program without slowing down compilation too much.



Code Generation

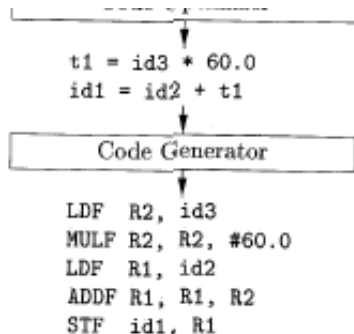
- The code generator takes as input an intermediate representation of the source program and maps it into the target language.
- If the target language is machine code, registers or memory locations are selected for each of the variables used by the program.
- Then, the intermediate instructions are translated into sequences of machine instructions that perform the same task.
- A crucial aspect of code generation is the judicious assignment of registers to hold variables.



Code Generation

- For example, using registers R1 and R2, the intermediate code might get translated into the machine code

```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```



Symbol-Table Management

- An essential function of a compiler is to record the variable names used in the source program and collect information about various attributes of each name.

1	position	...
2	initial	...
3	rate	...

SYMBOL TABLE

Symbol-Table Management

- These attributes may provide information about the storage allocated for a name, its type, its scope (where in the program its value may be used)
- In the case of procedure names, such things as the number and types of its arguments, the method of passing each argument (for example, by value or by reference), and the type returned.

1	position	...
2	initial	...
3	rate	...

SYMBOL TABLE

Symbol-Table Management

- The symbol table is a data structure containing a record for each variable name, with fields for the attributes of the name.
- The data structure should be designed to allow the compiler to find the record for each name quickly and to store or retrieve data from that record quickly.

1	position	...
2	initial	...
3	rate	...

SYMBOL TABLE

The Grouping of Phases into Passes

- The discussion of phases deals with the logical organization of a compiler.
- In an implementation, activities from several phases may be grouped together into a pass that reads an input file and writes an output file.
- For example, the front-end phases of lexical analysis, syntax analysis, semantic analysis, and intermediate code generation might be grouped together into one pass.
- Code optimization might be an optional pass.
- Then there could be a back-end pass consisting of code generation for a particular target machine.

The Grouping of Phases into Passes

- The discussion of phases deals with the logical organization of a compiler.
- In an implementation, activities from several phases may be grouped together into a pass that reads an input file and writes an output file.
- For example, the front-end phases of lexical analysis, syntax analysis, semantic analysis, and intermediate code generation might be grouped together into one pass.
- Code optimization might be an optional pass.
- Then there could be a back-end pass consisting of code generation for a particular target machine.

The Grouping of Phases into Passes — *continued*

- Some compiler collections have been created around carefully designed intermediate representations that allow the front end for a particular language to interface with the back end for a certain target machine.
- With these collections, we can produce compilers for different source languages for one target machine by combining different front ends with the back end for that target machine.
- Similarly, we can produce compilers for different target machines, by combining a front end with back ends for different target machines.

The Grouping of Phases into Passes — *continued*

- Some compiler collections have been created around carefully designed intermediate representations that allow the front end for a particular language to interface with the back end for a certain target machine.
- With these collections, we can produce compilers for different source languages for one target machine by combining different front ends with the back end for that target machine.
- Similarly, we can produce compilers for different target machines, by combining a front end with back ends for different target machines.

The Grouping of Phases into Passes — *continued*

- Some compiler collections have been created around carefully designed intermediate representations that allow the front end for a particular language to interface with the back end for a certain target machine.
- With these collections, we can produce compilers for different source languages for one target machine by combining different front ends with the back end for that target machine.
- Similarly, we can produce compilers for different target machines, by combining a front end with back ends for different target machines.

Compiler-Construction Tools

- The compiler writer, like any software developer, can profitably use modern software development environments containing tools such as language editors, debuggers, version managers, profilers, test harnesses, and so on.
- In addition to these general software-development tools, other more specialized tools have been created to help implement various phases of a compiler.

- These tools use specialized languages for specifying and implementing specific components, and many use quite sophisticated algorithms.
- The most successful tools are those that hide the details of the generation algorithm and produce components that can be easily integrated into the remainder of the compiler.

Some commonly used compiler-construction tools include

1. **Parser generators** that automatically produce syntax analyzers from a grammatical description of a programming language.
2. **Scanner generators** that produce lexical analyzers from a regular-expression description of the tokens of a language.
3. **Syntax-directed** translation engines that produce collections of routines for walking a parse tree and generating intermediate code.

Some commonly used compiler-construction tools include

4. **Code-generator generators** that produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine.
5. **Data-flow analysis engines** that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data-flow analysis is a key part of code optimization.
6. **Compiler-construction toolkits** that provide an integrated set of routines for constructing various phases of a compiler.

Can AI or LLMs do the task of a state-of-the-art compiler?

AI in Compiler Design

- Parts of the compilation process can be assisted by AI and LLM models.
- There is active research on using LLMs for compiler-related tasks.
- However, LLMs do not yet replace full traditional compilers.
- A compiler must be exact, deterministic, and semantics-preserving.
- LLMs can still generate invalid code, change semantics, or hallucinate.

- A major example is Large Language Models for Compiler Optimization by Cummins et al.
- It trained a 7B-parameter transformer for LLVM assembly and code-size optimization.
- The model predicts optimization options, instruction counts, and optimized code.
- The paper reports a 3.0% instruction-count improvement over the compiler.
- Generated code was compilable 91% of the time and perfectly emulated compiler output 70% of the time.

Recent Initiative: Meta LLM Compiler

- Meta introduced LLM Compiler, a family of models trained for compiler-related tasks.
- The models are built on Code Llama.
- They were trained on 546 billion tokens of LLVM IR and assembly code.
- The released model sizes include 7B and 13B parameters.
- The goal is to train LLMs on compiler representations, not only ordinary source code.

Meta LLM Compiler: Tasks

- Understanding LLVM IR.
- Understanding assembly.
- Optimizing code size.
- Predicting compiler behavior.

Compiler-Generated Feedback for LLMs

- Another recent direction uses the compiler itself to guide the LLM.
- The LLM proposes optimized IR, optimization passes, and instruction counts.
- The compiler checks whether the generated IR is compilable and whether predictions are correct.
- The feedback is sent back to the LLM for another attempt.

LLM proposes → Compiler checks → Feedback → LLM tries again

Recent Initiative: LLM4Decompile

- Decompile is the reverse of compilation:
machine code / assembly → high-level source code
- LLM4Decompile introduced open-source LLMs from 1.3B to 33B parameters trained for decompilation.
- The goal is to convert binary code into readable and executable C-like source code.
- This requires understanding assembly instructions, control flow, data flow, function structure, and high-level program reconstruction.

Recent AI Compiler Initiatives

- Large Language Models for Compiler Optimization
 - LLMs for LLVM assembly and optimization choices.
- Meta LLM Compiler
 - Code Llama-based models trained on LLVM IR and assembly.
- Compiler-generated feedback for LLMs
 - Compiler checks LLM-generated IR and provides feedback.
- LLM4Decompile
 - LLMs trained to decompile binaries into C-like code.